Teoria dos Grafos

Bruno Pereira dos Santos

Aula baseada nos cursos de outros docentes da UFMG (Prof. Antonio Alfredo Ferreira Loureiro), USP (Prof. Paulo Feofiloff), UFBA, livros, artigos e outros.

08 - Spanning Trees

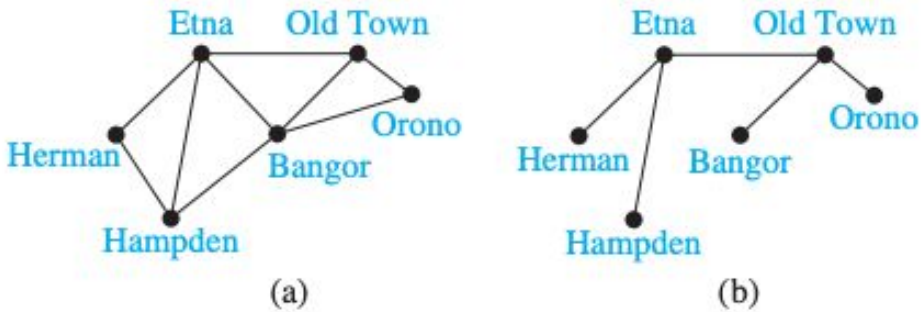
Árvores geradoras

Agenda

- Introdução
- Árvore geradora
  - Depth-First Search
  - Breadth-First Search
  - Aplicações
- Árvore geradora mínima
  - Prim
  - Kruskal

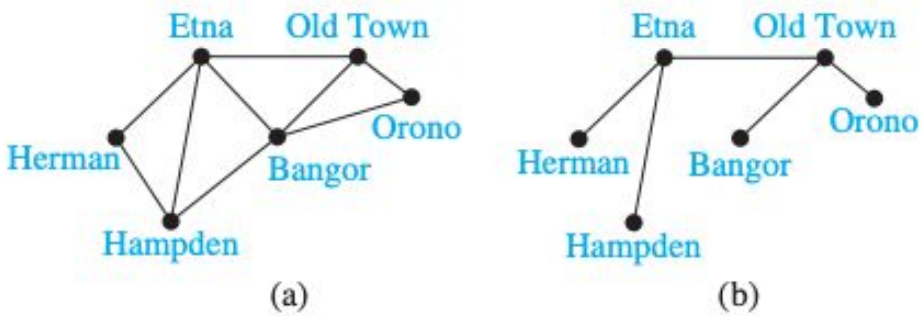
Introdução

- Considere um sistema de estradas conectando cidades
- Objetivo:
  - Manter todas as cidades conectadas
  - Usando o **menor número de estradas possível**
- Problema:
  - Quais estradas devem ser mantidas?



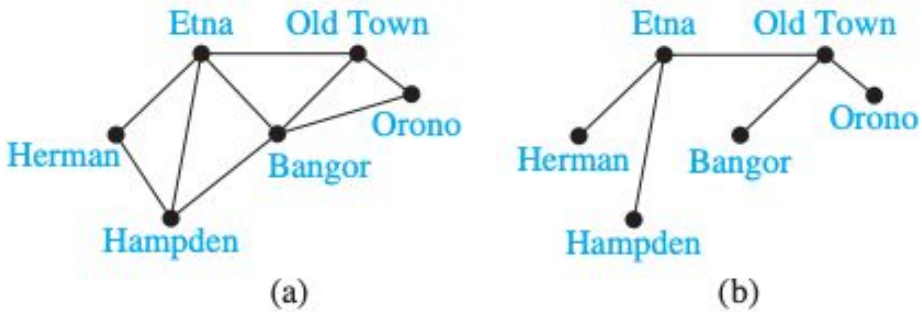
Introdução (1)

- Precisamos:
  - Garantir caminho entre quaisquer duas cidades
  - Minimizar o número de arestas
- Observação:
  - Um grafo com:
    - n vértices
    - $n-1$ arestas
    - Conectado
      - ➜ é uma árvore



Introdução (2)

- Solução:
  - Escolher um subgrafo conectado
  - Que contenha:
    - Todos os vértices
    - O menor número de arestas possível
    - ➜ Esse subgrafo é uma árvore

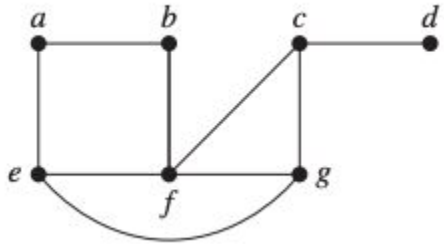


Introdução (3)

- Definição *Spanning Tree* (Árvore Geradora): Seja G um grafo simples. Uma *spanning tree* de G é um subgrafo de G que é:
  1. Uma árvore
  2. Contém todos os vértices de G
- Propriedades Importantes
  - Se um grafo possui spanning tree → ele é conectado
  - Recíproca:
    - Todo grafo simples conectado possui uma spanning tree

Exemplo

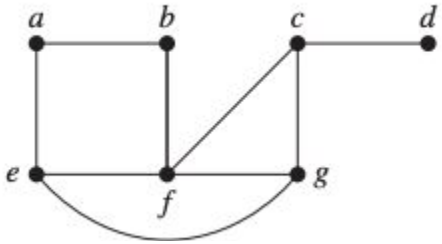
- Problema: Encontrar uma spanning tree do grafo G:

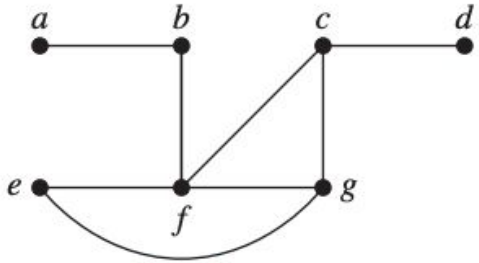


Produzindo uma árvore geradora de G removendo arestas que formam circuitos simples.

Exemplo

- Problema: Encontrar uma spanning tree do grafo G:



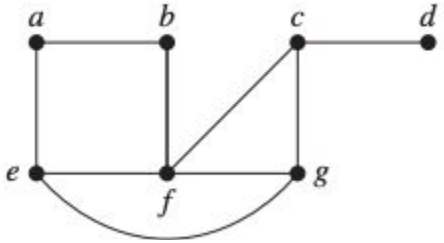


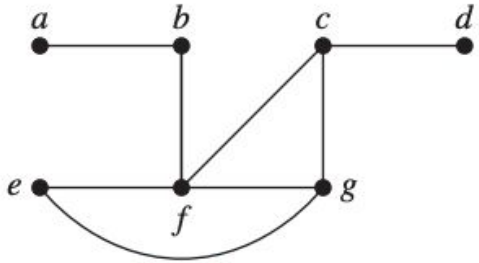
Edge removed: $\{a, e\}$

Produzindo uma árvore geradora de G removendo arestas que formam circuitos simples.

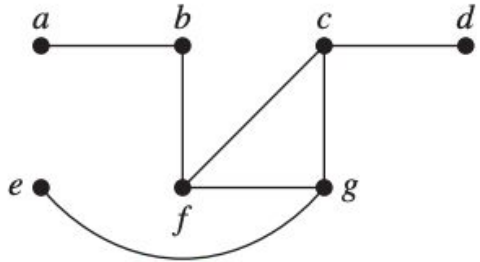
Exemplo

- Problema: Encontrar uma spanning tree do grafo G:





Edge removed: $\{a, e\}$

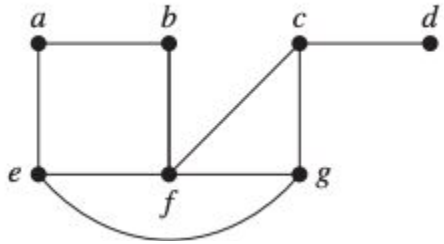


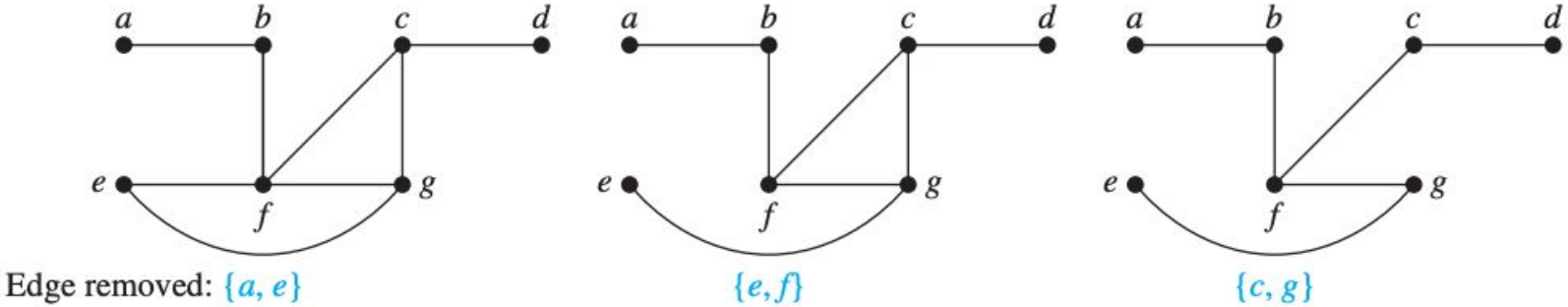
$\{e, f\}$

Produzindo uma árvore geradora de G removendo arestas que formam circuitos simples.

Exemplo

- Problema: Encontrar uma spanning tree do grafo G:

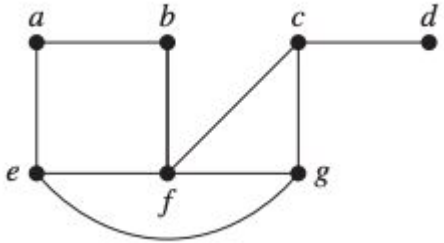


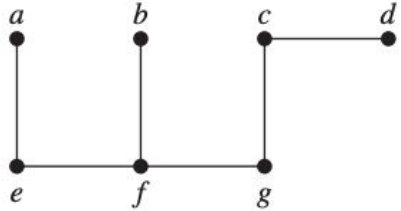


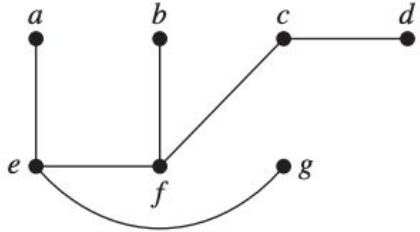
Produzindo uma árvore geradora de G removendo arestas que formam circuitos simples.

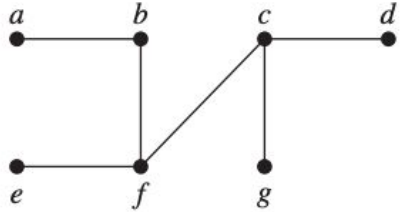
Exemplo (1)

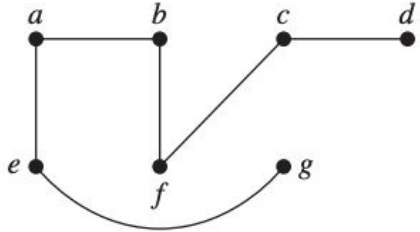
- Problema: Encontrar uma spanning tree do grafo G:





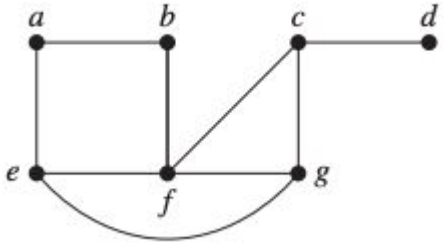


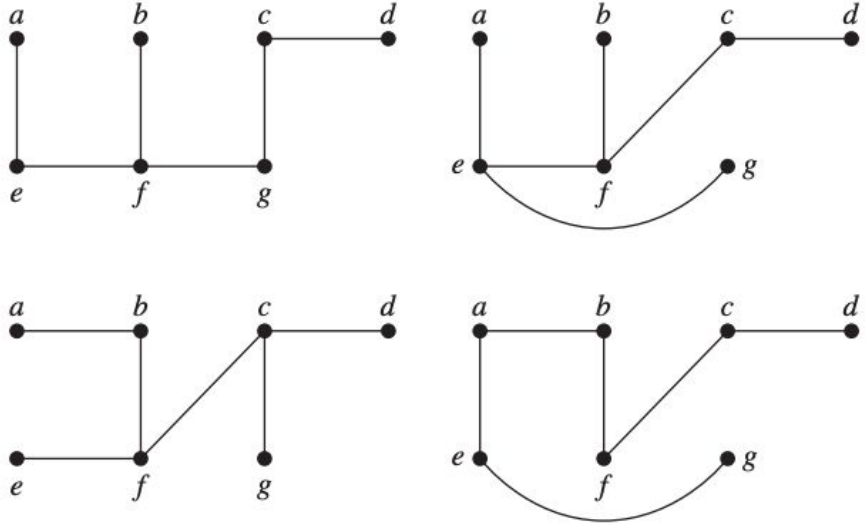




Exemplo (2)

- Problema: Encontrar uma spanning tree do grafo G:





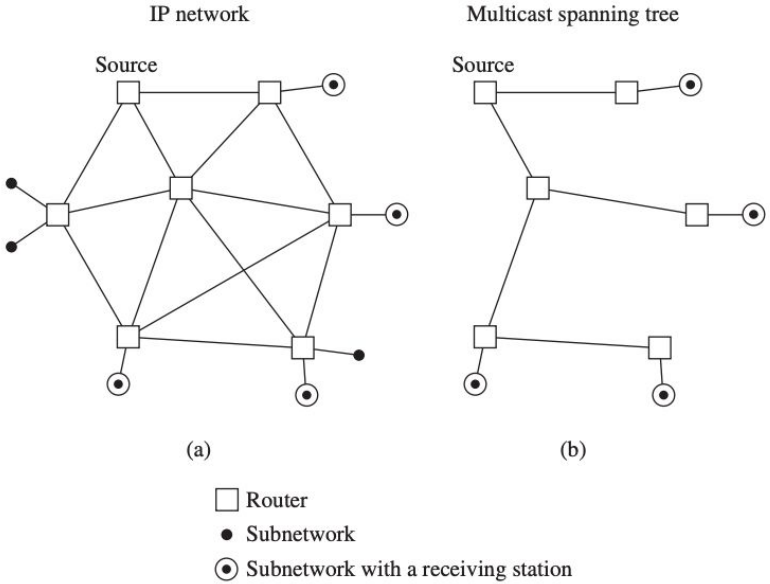
Árvores geradoras de G

Árvore geradora e Grafos simples

- Teorema: Um grafo simples é conectado se, e somente se, possui uma *spanning tree*
- Ideia da Prova (⇒): Se G tem uma *spanning tree* T:
  - T contém todos os vértices de G
  - Existe caminho entre quaisquer dois vértices em T
  - Como $T \subseteq G$:
    - Também existe caminho em G
  - ➜ Logo, G é conectado
- Ideia da Prova (⇐): Se G é conectado:
  - Pode conter ciclos
  - Remova uma aresta de um ciclo
  - Resultado:
    - Continua conectado
    - Tem menos arestas

Árvore geradora e Grafos simples (1)

- IP Multicasting
  - Árvores geradoras são usadas em redes IP (Internet Protocol)
  - Objetivo:
    - Enviar dados de uma fonte para múltiplos destinos
- Problema (Unicasting)
  - Enviar dados separadamente para cada destino:
    - Ineficiente
    - Muitas cópias da mesma informação
- Solução (Multicasting)
  - Enviar uma única cópia dos dados
  - Roteadores encaminham para outros nós
  - Resultado:
    - Todos os destinos recebem os dados
    - Uso eficiente da rede
- Problema dos Ciclos
  - Dados não devem:
    - Voltar para o mesmo roteador
    - Circular em loops
  - Em grafos:
    - Isso significa evitar ciclos



Depth-First Search (DFS)

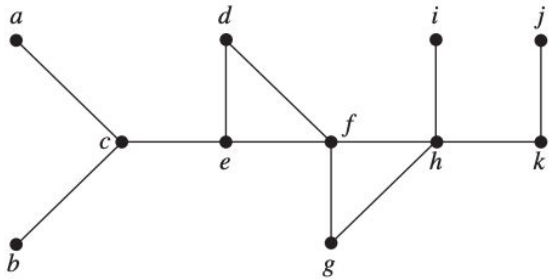
- Método para construir *spanning trees*
- Alternativa mais eficiente que remover ciclos (teorema 1)
- Ideia Principal
  - Em vez de remover arestas:
    - Construir a árvore adicionando arestas
  - Base:
    - Explorar o grafo em profundidade (depth-first)
- Como Funciona (Visão Geral)
  - Escolher um vértice inicial (raiz)
  - Construir um caminho:
    - Adicionando vértices não visitados
  - Continuar o máximo possível

Depth-First Search (DFS) (1)

- Expansão do Caminho
  - Sempre:
    - Escolher um vizinho não visitado
  - Adicionar:
    - Aresta + vértice
  - ➜ Forma um caminho longo "profundo" no grafo
- Quando não dá para avançar
  - Se não houver vértices não visitados:
    - Voltar (backtracking)
  - Retornar ao vértice anterior
  - Tentar outro caminho
- Processo
  - Repetir:
    - Avançar o máximo possível
    - Voltar quando necessário
  - Continua até:
    - Todos os vértices serem visitados
- Produz:
  - Uma *spanning tree*
  - Propriedades:
    - Conectada
    - Sem ciclos
    - Contém todos os vértices
- Características Importantes
  - Pode ser recursivo ou interativo
  - Também chamada de (se na versão recursiva):
    - Backtracking
  - Classificação dos vértices:
    - Folhas → fim de caminhos
    - Internos → pontos de expansão

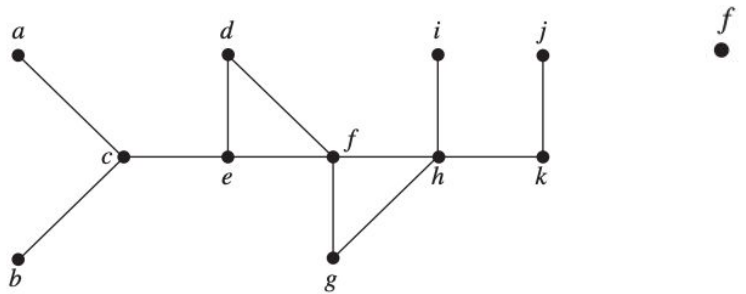
Depth-First Search (DFS) (2)

- Problema: Usar Depth-First Search (DFS) para encontrar uma spanning tree do grafo G mostrado abaixo.



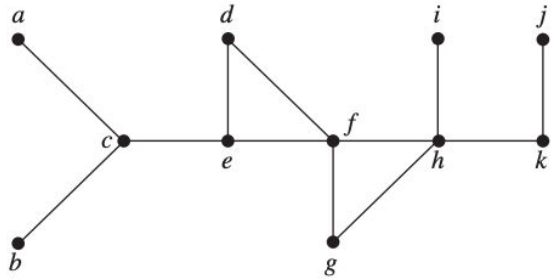
Depth-First Search (DFS) (3)

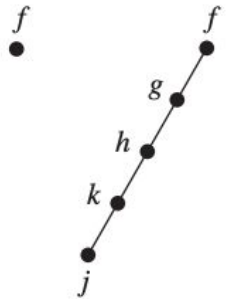
- Problema: Usar Depth-First Search (DFS) para encontrar uma spanning tree do grafo G mostrado abaixo.



Depth-First Search (DFS) (4)

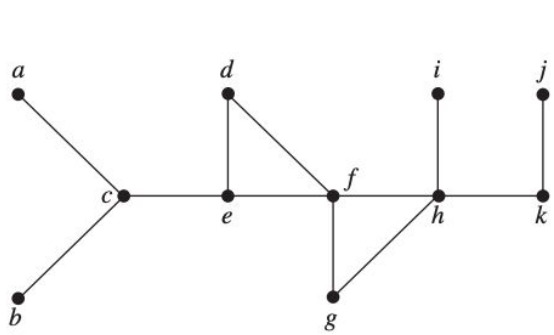
- Problema: Usar Depth-First Search (DFS) para encontrar uma spanning tree do grafo G mostrado abaixo.



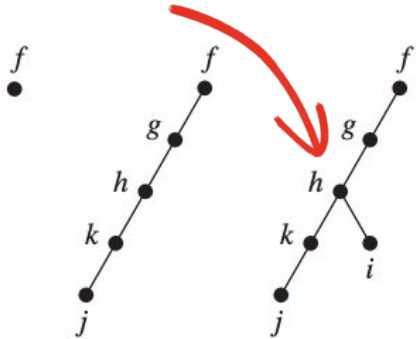


Depth-First Search (DFS) (5)

- Problema: Usar Depth-First Search (DFS) para encontrar uma spanning tree do grafo G mostrado abaixo.

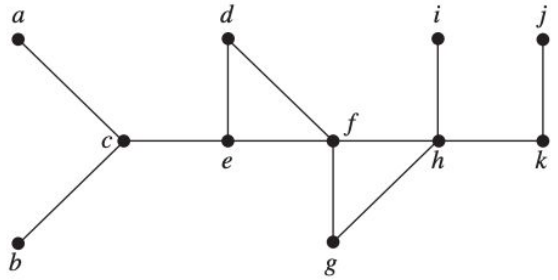


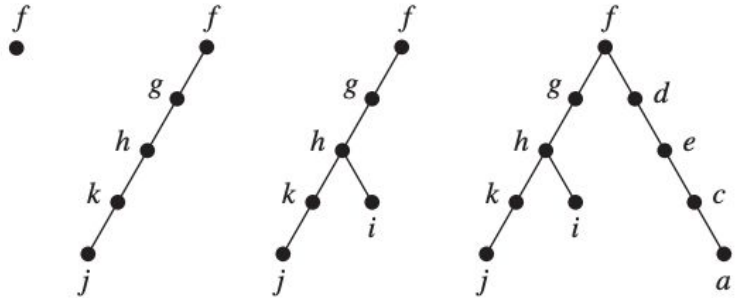
BACKTRACKING



Depth-First Search (DFS) (6)

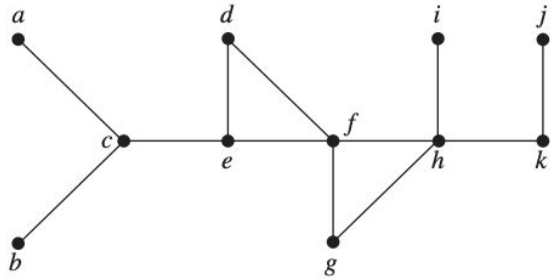
- Problema: Usar Depth-First Search (DFS) para encontrar uma spanning tree do grafo G mostrado abaixo.

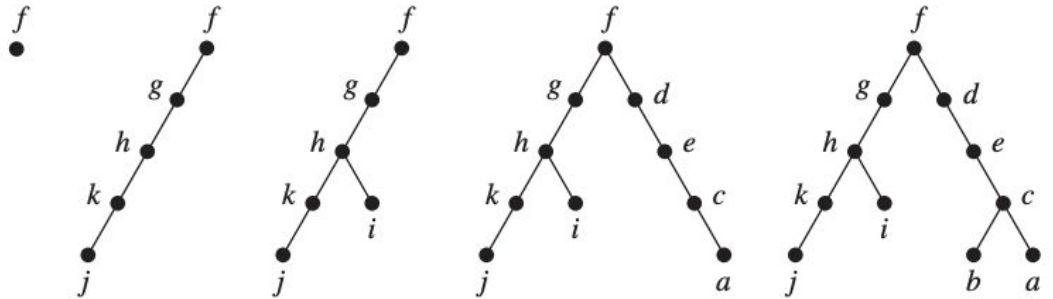




Depth-First Search (DFS) (7)

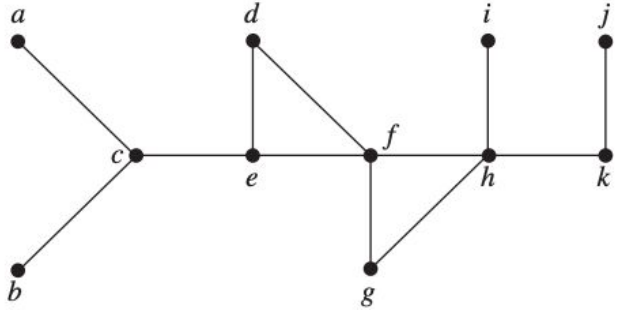
- Problema: Usar Depth-First Search (DFS) para encontrar uma spanning tree do grafo G mostrado abaixo.

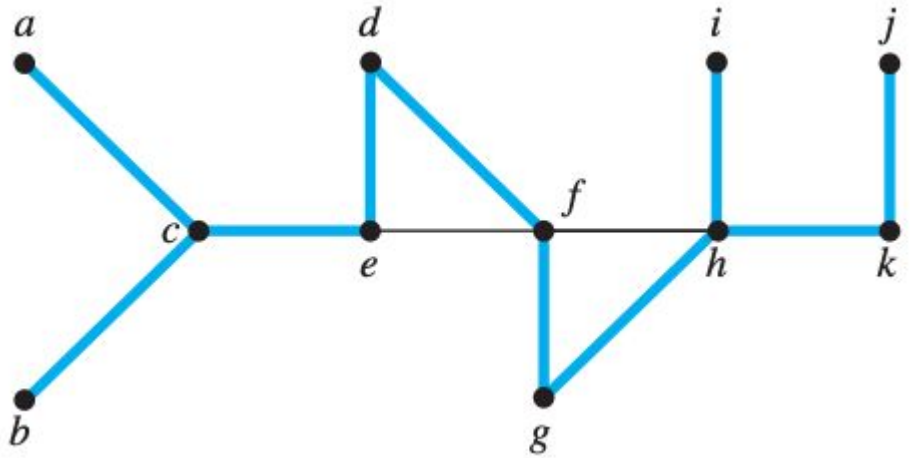




Depth-First Search (DFS) (8)

- Problema: Usar Depth-First Search (DFS) para encontrar uma spanning tree do grafo G mostrado abaixo.





Depth-First Search (DFS) (9)

- Construímos uma spanning tree adicionando vértices e arestas
- Escolhemos um vértice inicial v_1 como raiz
- Inicialmente: T contém apenas v_1
- Construção da Árvore
  - A cada passo:
    - Adicionar um novo vértice w
    - Conectar com um vértice já em T
  - Explorar recursivamente a partir de w
- Propriedades
  - Não cria ciclos:
    - Nunca adiciona aresta entre vértices já na árvore
  - Mantém conectividade
  - Como G é conectado:
    - Todos os vértices são visitados
  - ➜ Resultado: spanning tree
- Complexidade
  - Cada vértice é visitado uma única vez
  - Cada aresta é examinada no máximo duas vezes
  - O(E) ou O(V^2)

ALGORITHM 1 Depth-First Search.

procedure *DFS*(G: connected graph with vertices $v_1, v_2, \ldots, v_n$)
$T :=$ tree consisting only of the vertex v_1
visit(v_1)

procedure *visit*(v: vertex of G)
for each vertex w adjacent to v and not yet in T
  add vertex w and edge $\{v, w\}$ to T
  visit(w)

Depth-First Search (DFS) (10)

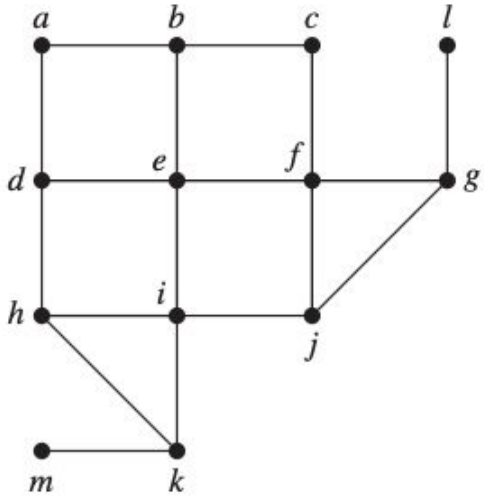
- Aplicações do DFS
  - Encontrar:
    - Caminhos
    - Ciclos
  - Determinar:
    - Componentes conexas
  - Identificar:
    - Vértices de corte

Breadth-First Search (BFS)

- A BFS pode ser usada para construir uma árvore geradora de um grafo simples.
- Ideia Principal
  - Escolha arbitrariamente um vértice raiz.
    - Adicione todas as arestas incidentes a esse vértice → formam o nível 1.
  - Para cada vértice do nível atual:
    - Adicione arestas incidentes que não formem ciclos.
    - Os novos vértices formam o próximo nível.
  - Repita o processo nível a nível até incluir todos os vértices.
- Observações:
  - O processo termina pois o grafo tem número finito de arestas.
  - O resultado é uma árvore geradora contendo todos os vértices do grafo.

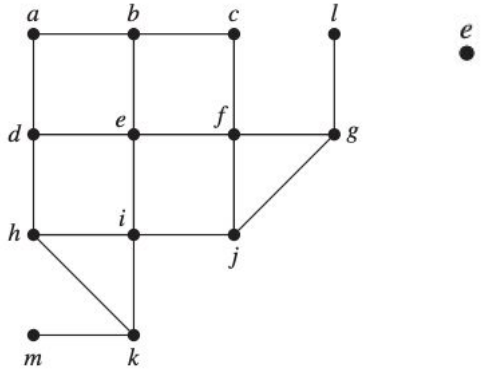
Breadth-First Search (BFS) (1)

- Problema: Usar *Breadth-First Search (BFS)* para encontrar uma spanning tree do grafo G mostrado abaixo.



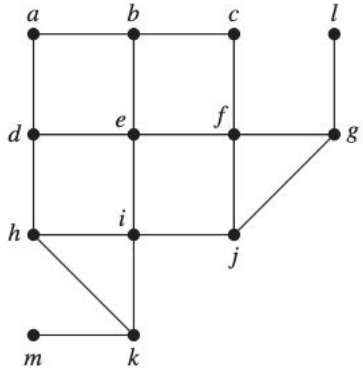
Breadth-First Search (BFS) (2)

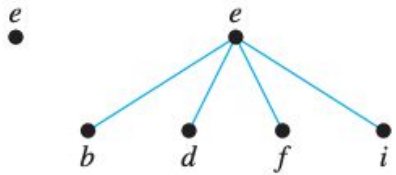
- Problema: Usar *Breadth-First Search (BFS)* para encontrar uma spanning tree do grafo G mostrado abaixo.



Breadth-First Search (BFS) (3)

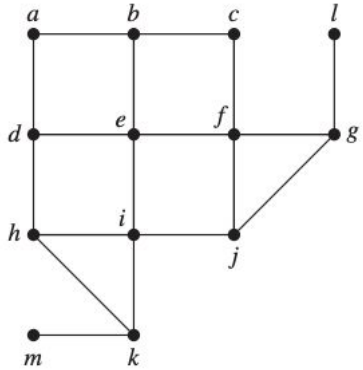
- Problema: Usar *Breadth-First Search (BFS)* para encontrar uma spanning tree do grafo G mostrado abaixo.

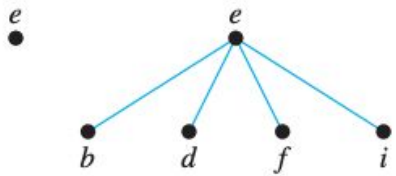


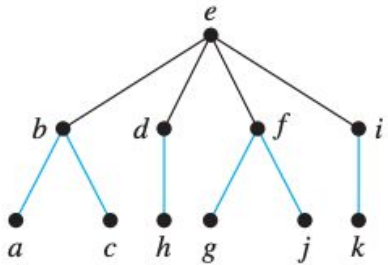


Breadth-First Search (BFS) (4)

- Problema: Usar *Breadth-First Search (BFS)* para encontrar uma spanning tree do grafo G mostrado abaixo.

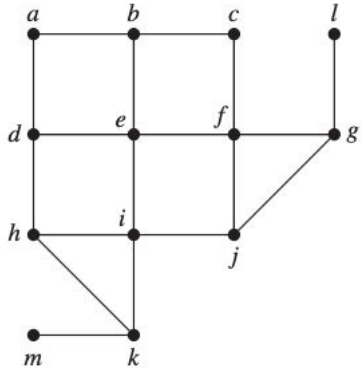


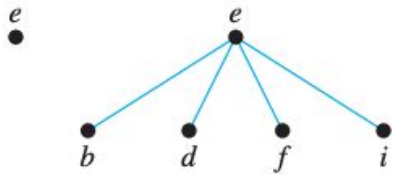


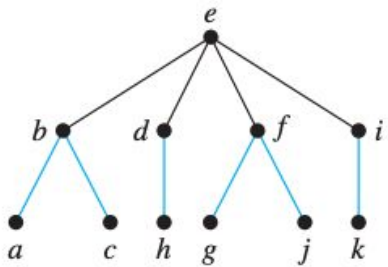


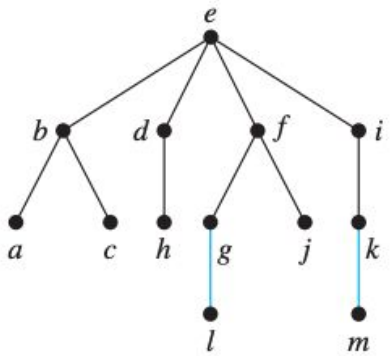
Breadth-First Search (BFS) (5)

- Problema: Usar *Breadth-First Search (BFS)* para encontrar uma spanning tree do grafo G mostrado abaixo.









Breadth-First Search (BFS) (6)

- Ideia Geral
  - Explora o grafo nível por nível a partir de um vértice inicial
  - Evita ciclos ao não revisitar vértices já incluídos
  - Constrói uma árvore geradora
- Complexidade
  - Cada aresta é examinada no máximo duas vezes
  - O(E) ou $O(V^2)$

ALGORITHM 2 Breadth-First Search.

procedure *BFS*(G: connected graph with vertices $v_1, v_2, \ldots, v_n$)
T := tree consisting only of vertex v_1
L := empty list
put v_1 in the list L of unprocessed vertices
while L is not empty
  remove the first vertex, v, from L
  for each neighbor w of v
    if w is not in L and not in T **then**
      add w to the end of the list L
      add w and edge $\{v, w\}$ to T

DFS vs BFS

- Ambos constroem árvores geradoras em grafos conexos
- Podem resolver os mesmos problemas, mas com abordagens diferentes

Características	BFS (Busca em Largura)	DFS (Busca em Profundidade)
Estratégia	Nível por nível	Vai até o fundo primeiro
Estrutura de dados	Fila (queue)	Pilha (stack) / recursão
Ordem de exploração	Vértices mais próximos primeiro	Caminhos mais profundos primeiro
Caminho mínimo	Garante (em nº de arestas)	Não garante
Uso de memória	Maior (armazenar níveis)	Menor (em geral)
Aplicações típicas	Menor caminho, níveis, bipartição	Componentes, ciclos, backtracking
Grafos densos	Pode ser custoso	Pode ser mais eficiente
Grafos esparsos	Geralmente eficiente	Também eficiente
Intuição	Expansão em “ondas”	Exploração em “profundidade”

Aplicações de *Backtracking*

- Usado para resolver problemas com busca exaustiva
- Explora todas as possíveis soluções de forma sistemática
- Árvore de Decisão
  - Cada vértice interno → uma decisão
  - Cada folha → uma possível solução
  - Uma sequência de decisões forma um caminho na árvore
- Como Funciona
  1. Constrói uma sequência de decisões até onde for possível
  2. Se não houver solução:
     - Retrocede (backtrack) ao vértice anterior
  3. Tenta um novo caminho
  4. Repete até:
     - Encontrar uma solução, ou
     - Concluir que não existe solução

Aplicações de *Backtracking* (1)

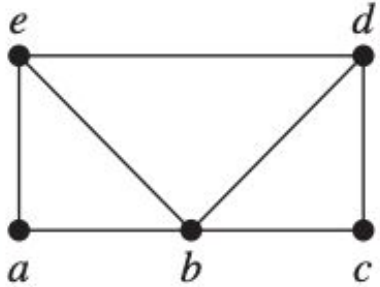
- Problema: Determinar se um grafo pode ser colorido com n cores? (vértices adjacentes não podem ter a mesma cor)
- Ideia do *Backtracking*
  - Atribuir cores aos vértices sequencialmente
  - Sempre usar a primeira cor disponível
  - Se não for possível colorir um vértice:
    - Retroceder (*backtrack*) e tentar outra cor

Aplicações de *Backtracking* (2)

- Problema: Determinar se um grafo pode ser colorido com n cores? (vértices adjacentes não podem ter a mesma cor)
- Passos do Algoritmo
  1. Escolha um vértice inicial e atribua uma cor
  2. Para cada próximo vértice:
     - Tente uma cor válida
     - Se não houver cor possível → *backtrack*
  3. Continue até:
     - Colorir todos os vértices, ou
     - Concluir que não há solução

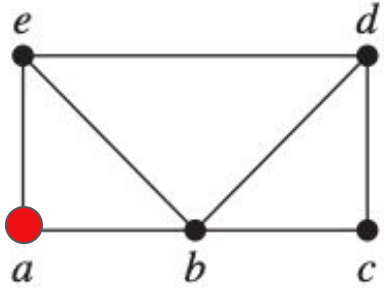
Aplicações de *Backtracking* (3)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green



Aplicações de *Backtracking* (4)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green

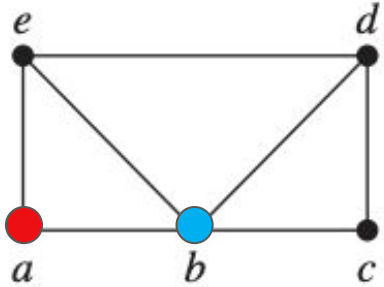


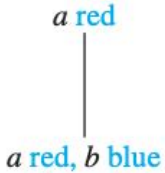
a red

1. a → vermelho

Aplicações de *Backtracking* (5)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green

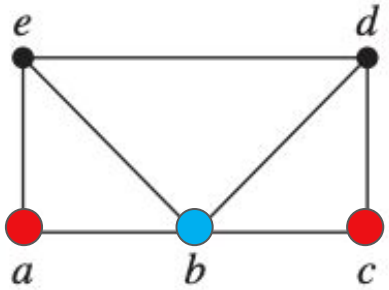


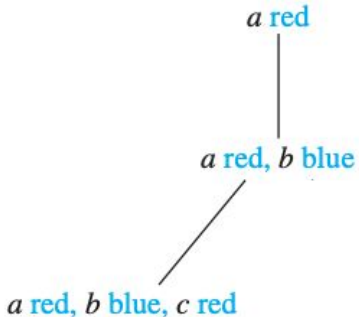


1. a → vermelho
2. b → azul

Aplicações de *Backtracking* (6)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green

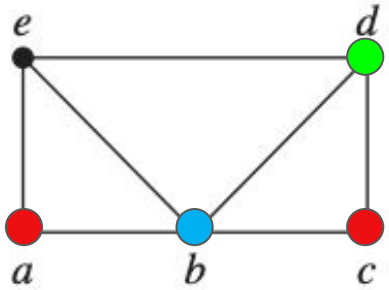




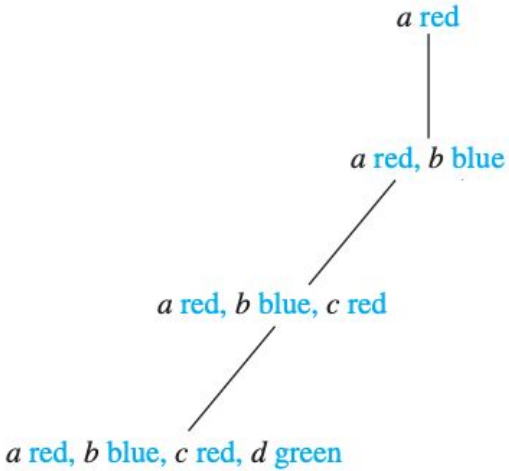
1. a → vermelho
2. b → azul
3. c → vermelho

Aplicações de *Backtracking* (7)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green

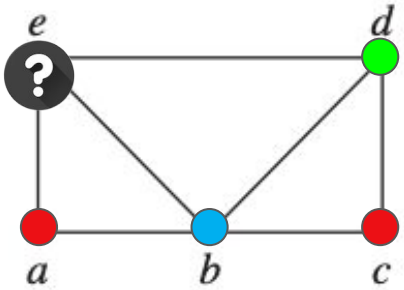


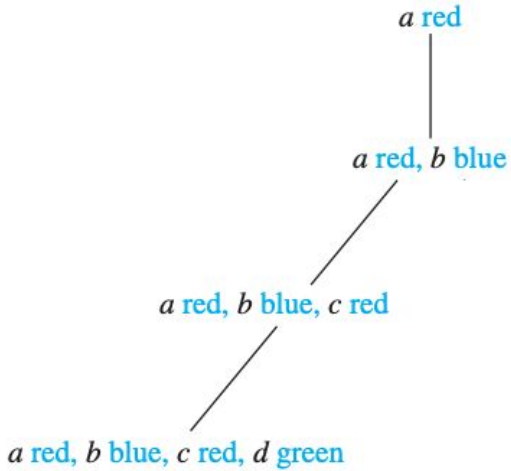
1. a → vermelho
2. b → azul
3. c → vermelho
4. d → verde



Aplicações de *Backtracking* (8)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green

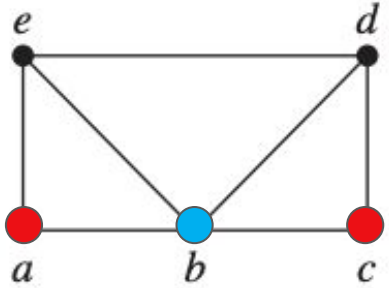


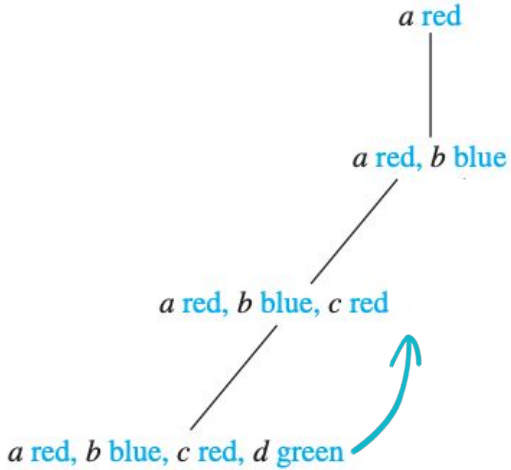


1. a → vermelho
2. b → azul
3. c → vermelho
4. d → verde
5. Tentar colorir e → **falha** (nenhuma cor disponível)

Aplicações de *Backtracking* (9)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green



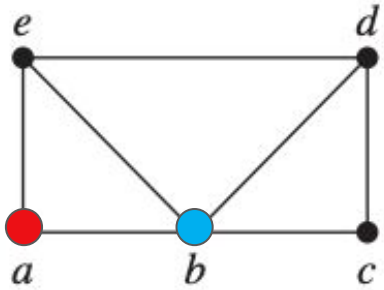


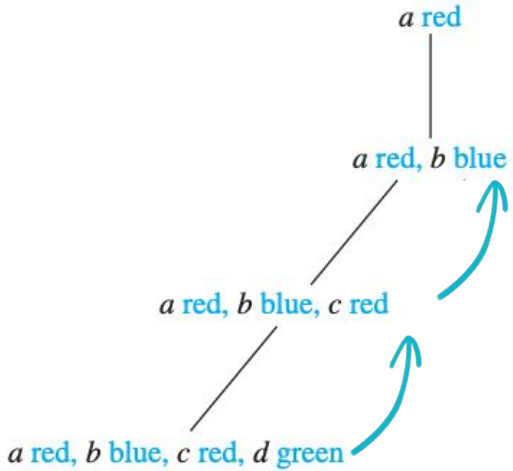
1. a → vermelho
2. b → azul
3. c → vermelho
4. d → verde
5. Tentar colorir e → **falha** (nenhuma cor disponível)
6. **Backtracking:**

- Volta para d → não há alternativa

Aplicações de *Backtracking* (10)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green



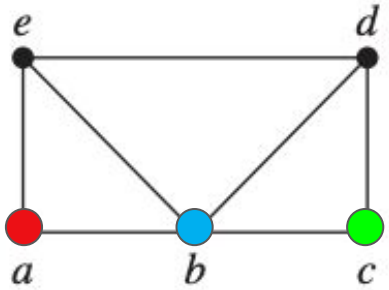


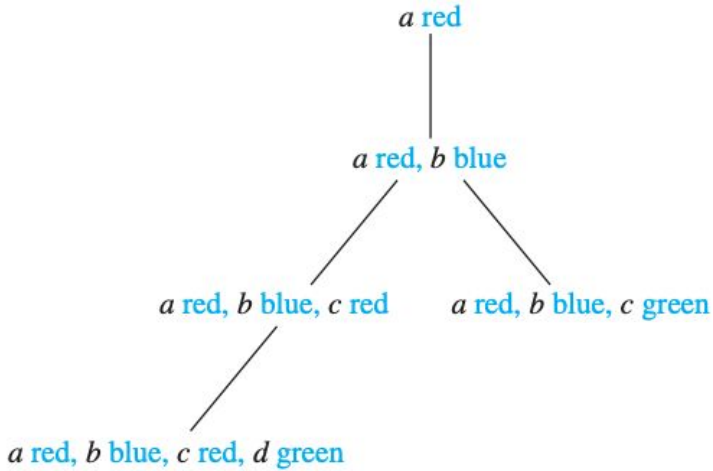
1. a → vermelho
2. b → azul
3. c → vermelho
4. d → verde
5. Tentar colorir e → **falha** (nenhuma cor disponível)
6. **Backtracking:**

- Volta para d → não há alternativa
- Volta para c

Aplicações de *Backtracking* (11)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green



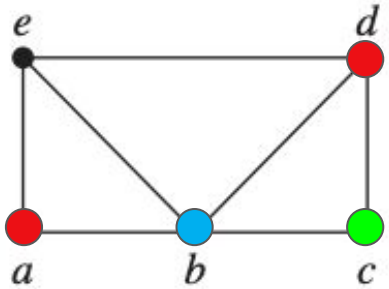


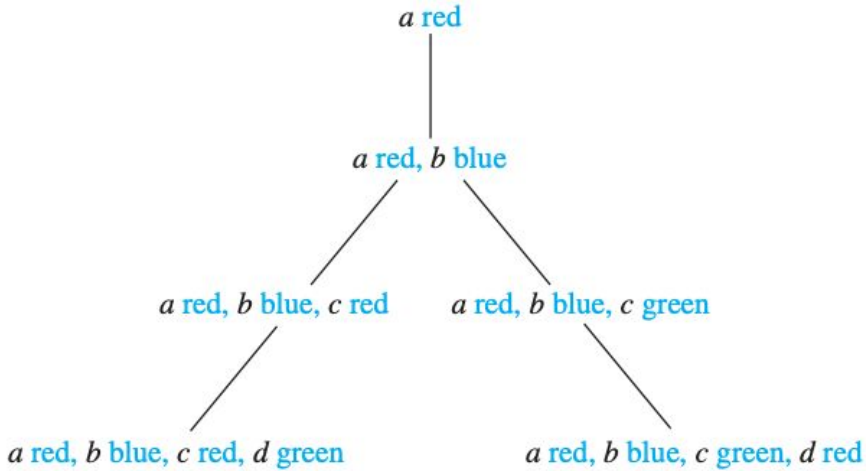
1. a → vermelho
2. b → azul
3. c → vermelho
4. d → verde
5. Tentar colorir e → **falha** (nenhuma cor disponível)
6. **Backtracking:**

- Volta para d → não há alternativa
- Volta para c → troca para **verde**

Aplicações de *Backtracking* (12)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green

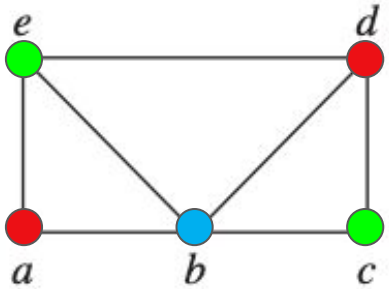




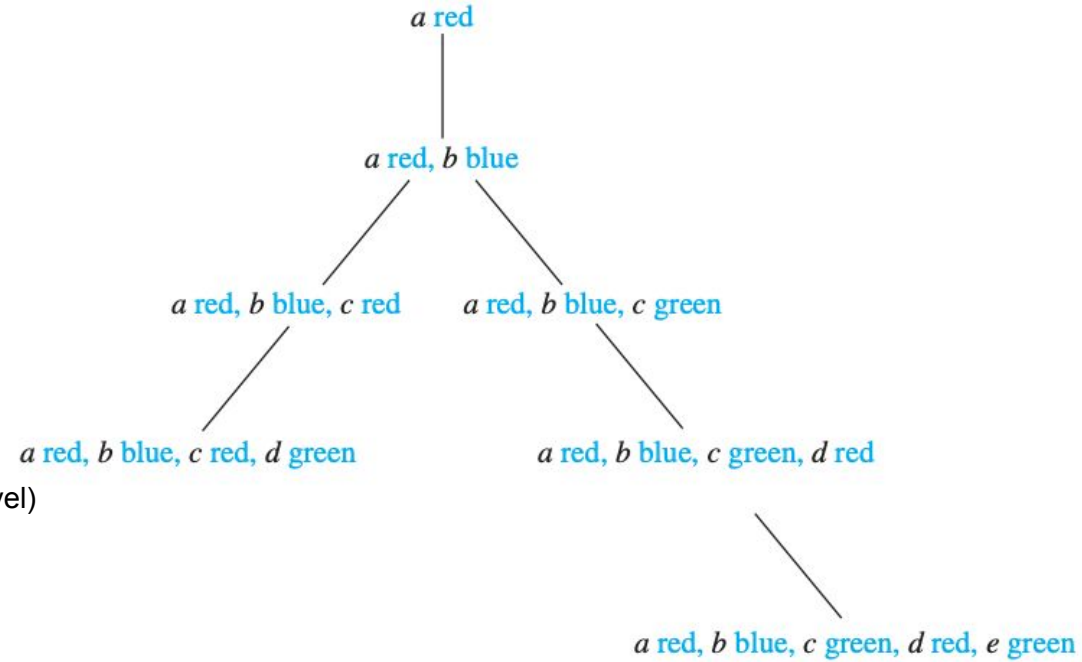
1. a → vermelho
2. b → azul
3. c → vermelho
4. d → verde
5. Tentar colorir e → **falha** (nenhuma cor disponível)
6. **Backtracking 2x**
7. d → vermelho

Aplicações de *Backtracking* (13)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green

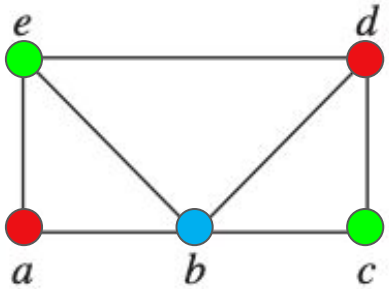


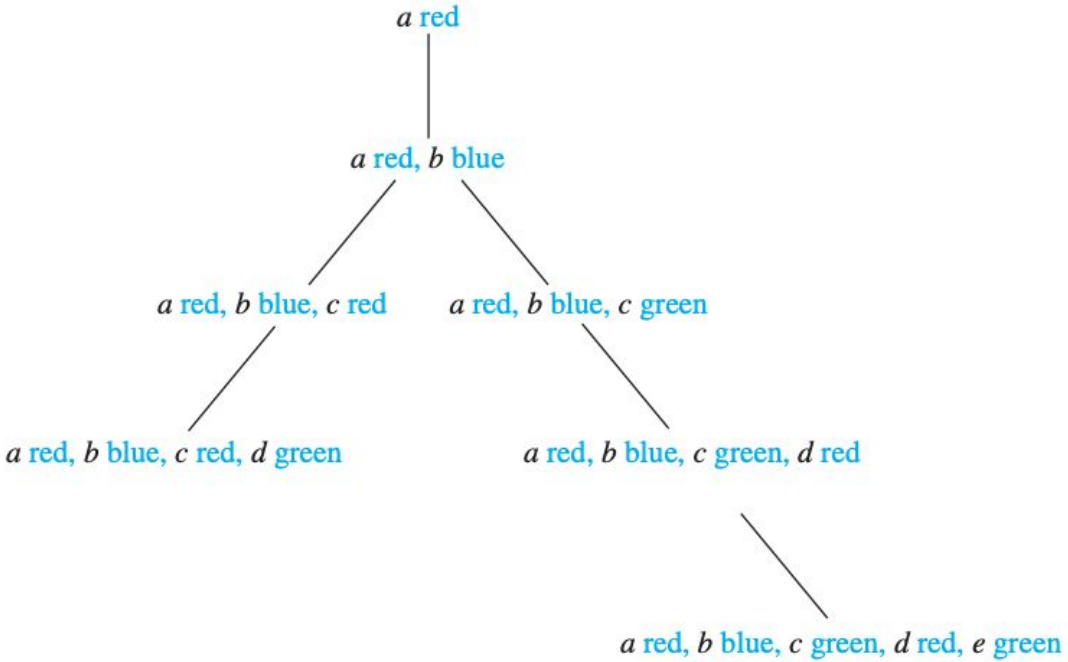
1. a → vermelho
2. b → azul
3. c → vermelho
4. d → verde
5. Tentar colorir e → **falha** (nenhuma cor disponível)
6. **Backtracking 2x**
7. d → vermelho
8. e → verde



Aplicações de *Backtracking* (14)

- Exemplo: Colorir o grafo com 3 cores: red, blue e green





- Ramos = tentativas de coloração
- Folhas = soluções ou falhas
- Backtracking evita explorar completamente caminhos inválidos

Aplicações de *Backtracking* (15)

- Árvore de Decisão
  - Cada nível representa a coloração de um vértice
  - Cada caminho representa uma possível solução
  - Backtracking evita explorar caminhos inválidos até o fim

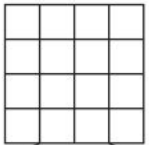
Aplicações de *Backtracking* (16)

- Problema: Posicionar n rainhas em um tabuleiro $n \times n$
  - Restrição: nenhuma rainha pode atacar outra
    - Não podem estar na mesma:
      - Linha
      - Coluna
      - Diagonal
- Ideia com Backtracking
  - Construímos a solução coluna por coluna
  - Em cada etapa, tentamos colocar uma rainha em uma posição válida
- Processo
  1. Comece com tabuleiro vazio
  2. Coloque uma rainha na próxima coluna
  3. Verifique:
     - Linha livre
     - Diagonais livres
  4. Se não houver posição válida → backtrack
  5. Continue até completar ou falhar

Aplicações de *Backtracking* (17)

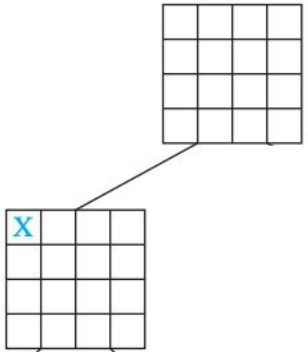
- Problema: Posicionar n rainhas em um tabuleiro $n \times n$
  - Estratégia do Algoritmo
    - Etapa k: já existem rainhas nas k primeiras colunas
    - Tentar posicionar na coluna k+1
    - Percorrer linhas de cima para baixo
  - Se falhar:
    - Voltar para coluna anterior
    - Tentar próxima posição possível

Aplicações de *Backtracking* (18)



- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível

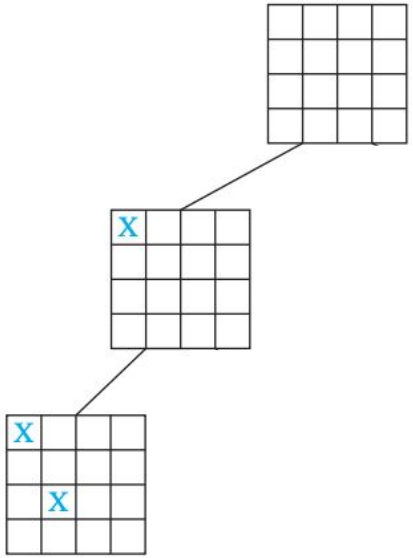
Aplicações de *Backtracking* (19)



- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível

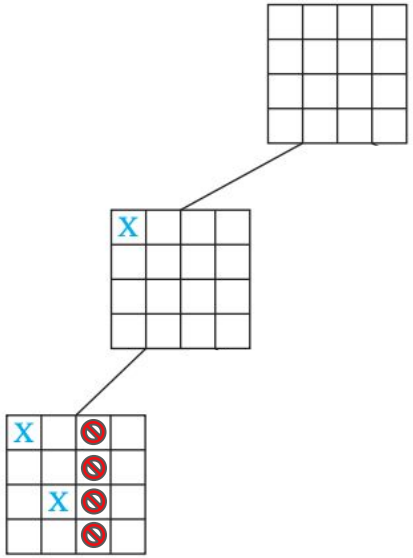
Aplicações de *Backtracking* (20)

- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível



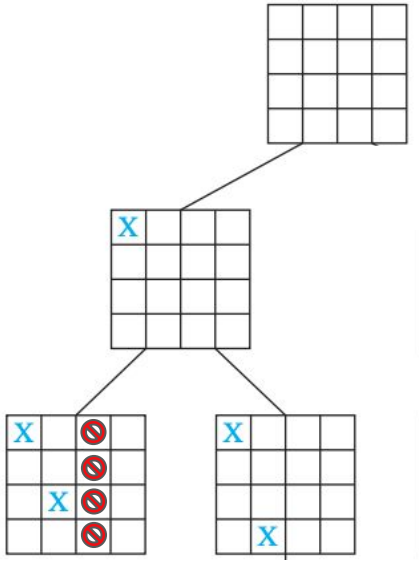
Aplicações de *Backtracking* (21)

- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível



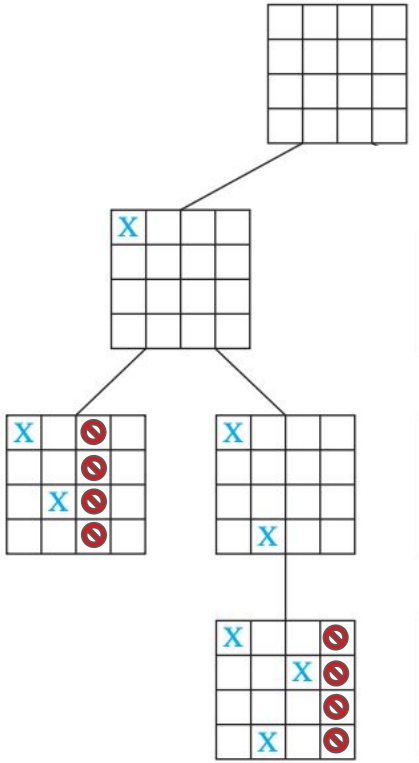
Aplicações de *Backtracking* (22)

- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível



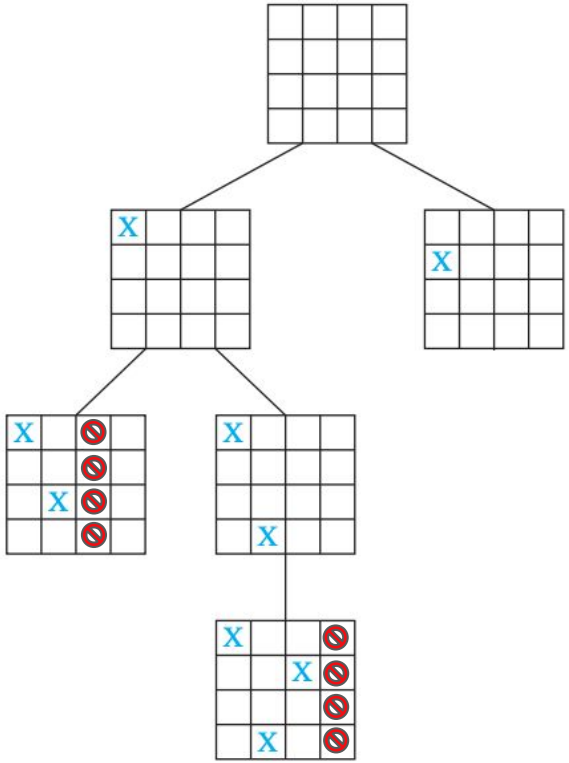
Aplicações de *Backtracking* (23)

- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível



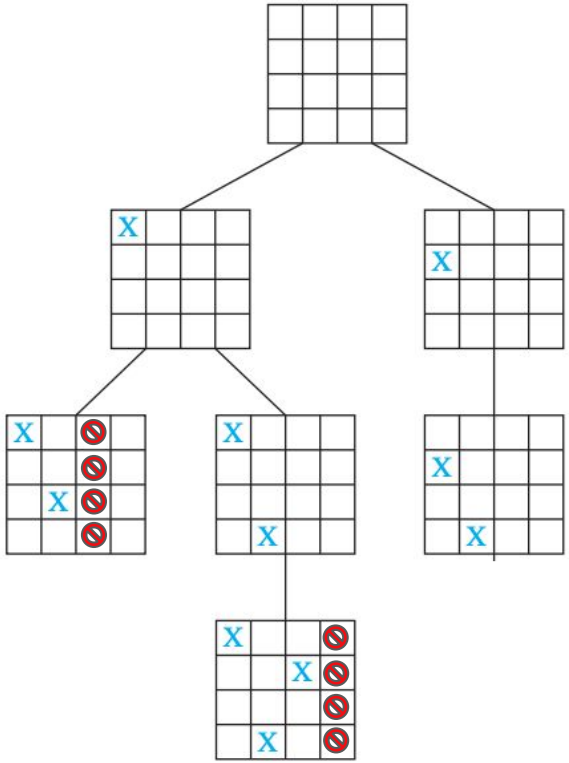
Aplicações de *Backtracking* (24)

- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível



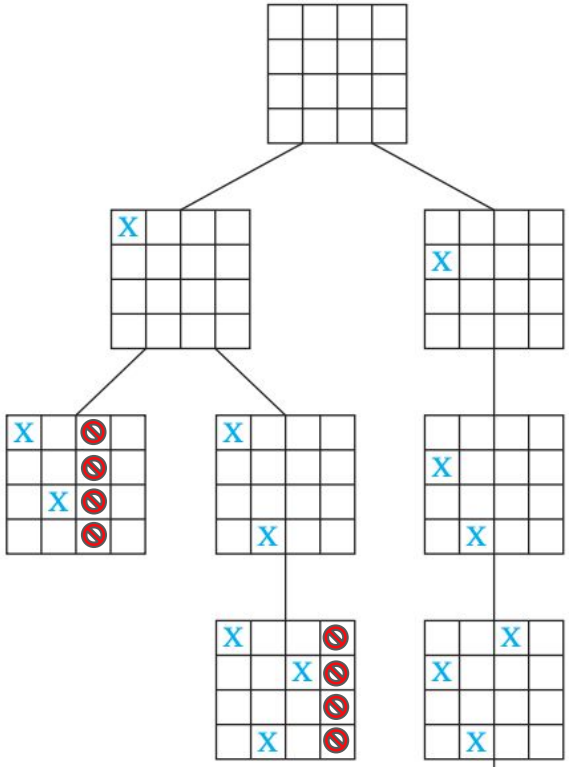
Aplicações de *Backtracking* (25)

- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível



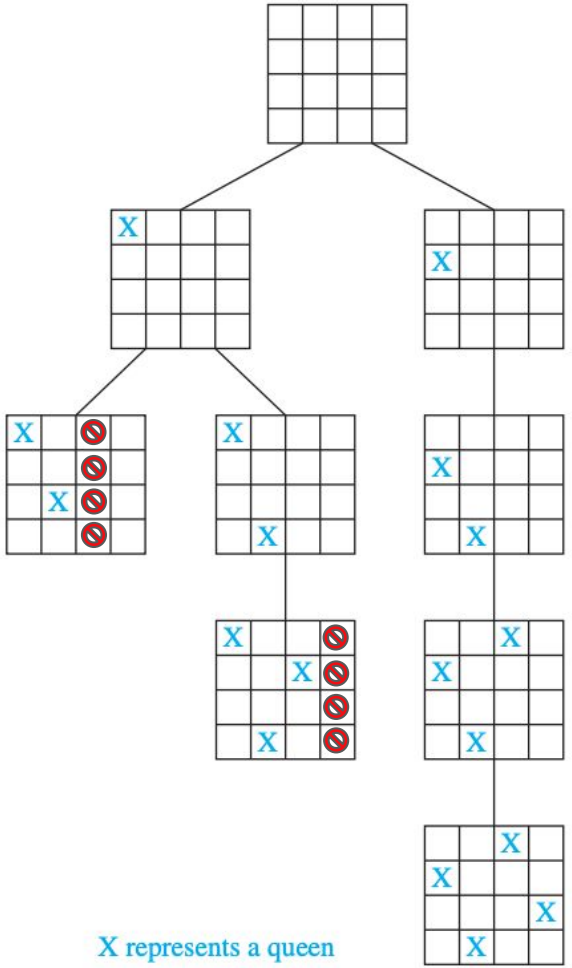
Aplicações de *Backtracking* (26)

- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível



Aplicações de *Backtracking* (27)

- Estratégia do Algoritmo
  - Etapa k: já existem rainhas nas k primeiras colunas
  - Tentar posicionar na coluna k+1
  - Percorrer linhas de cima para baixo
- Se falhar:
  - Voltar para coluna anterior
  - Tentar próxima posição possível

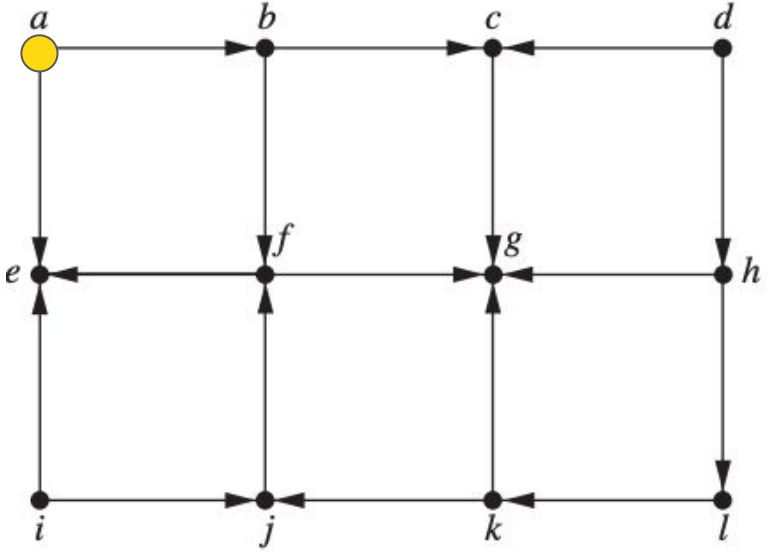


DFS/BFS em grafos direcionados

- BFS e DFS podem ser adaptados para grafos dirigidos
- A saída não é necessariamente uma árvore geradora, mas uma **floresta geradora (*spanning forest*)**
- Só adicionamos uma aresta (v, w) se:
  - Ela está orientada de v para w
  - w ainda não foi visitado
- Construção da Floresta
  - Executamos DFS (ou BFS) normalmente
  - Quando não há mais arestas saindo para vértices não visitados:
    - Escolhemos um novo vértice não visitado
    - Ele se torna a raiz de uma nova árvore
- Resultado
  - Em vez de uma única árvore:
    - Obtemos uma floresta de árvores
  - Cada árvore corresponde a uma parte alcançável do grafo

DFS/BFS em grafos direcionados (1)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

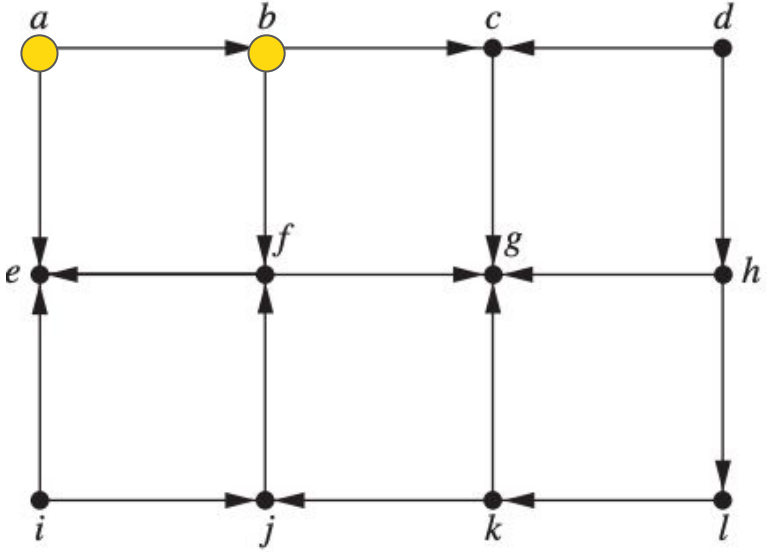




Visitado
Em visitação

DFS/BFS em grafos direcionados (2)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

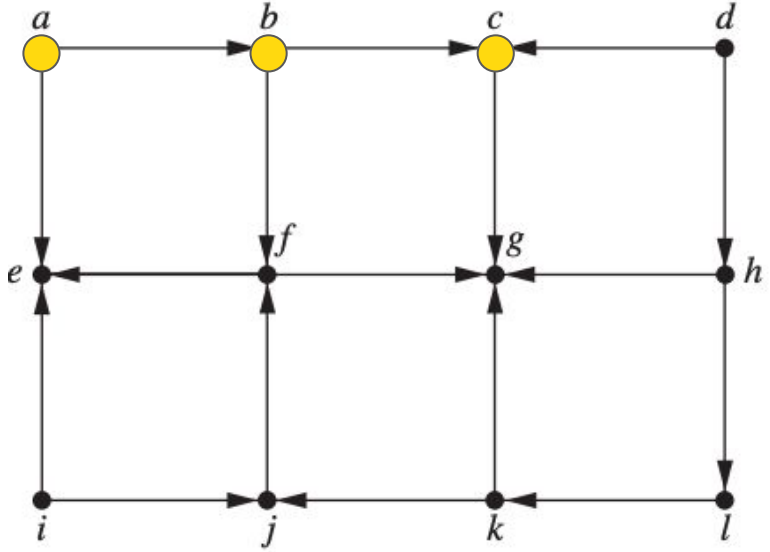


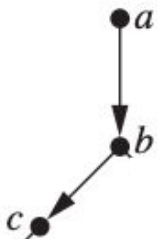


Visitado
Em visitação

DFS/BFS em grafos direcionados (3)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

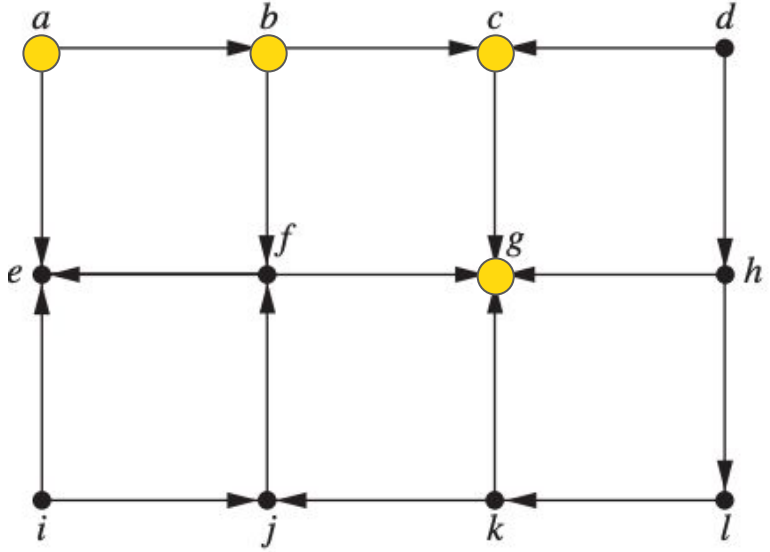


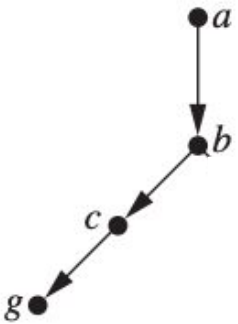


Visitado
Em visitação

DFS/BFS em grafos direcionados (4)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

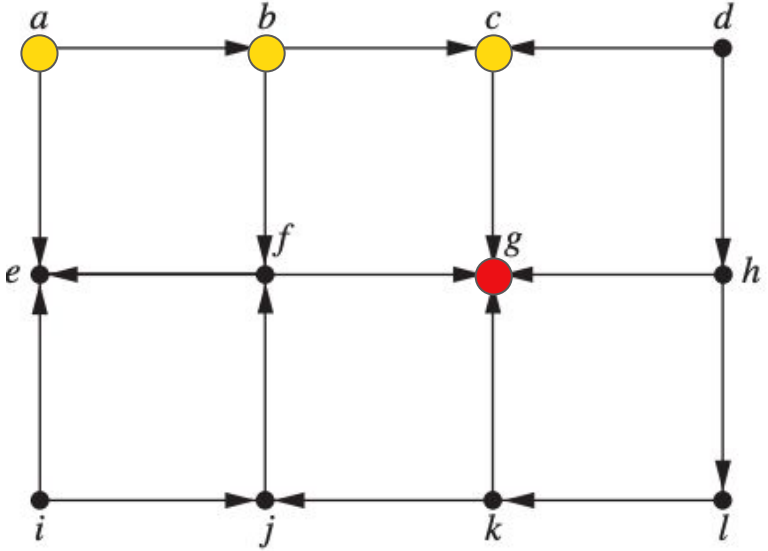


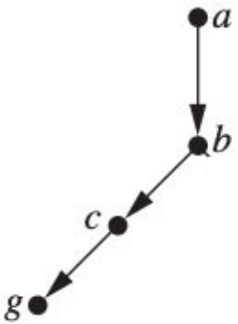


Visitado
Em visitação

DFS/BFS em grafos direcionados (5)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

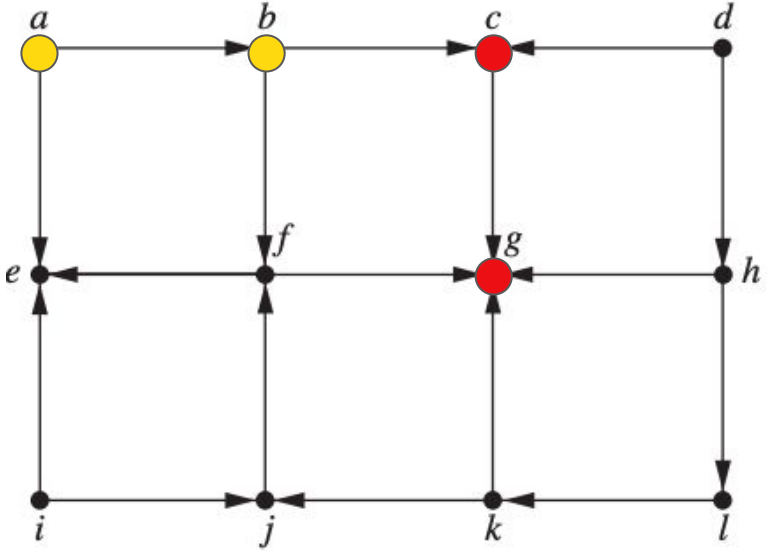


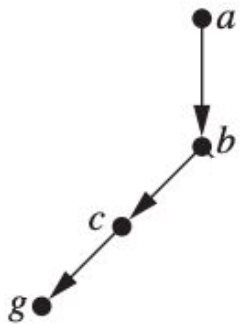


Visitado
Em visitação

DFS/BFS em grafos direcionados (6)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

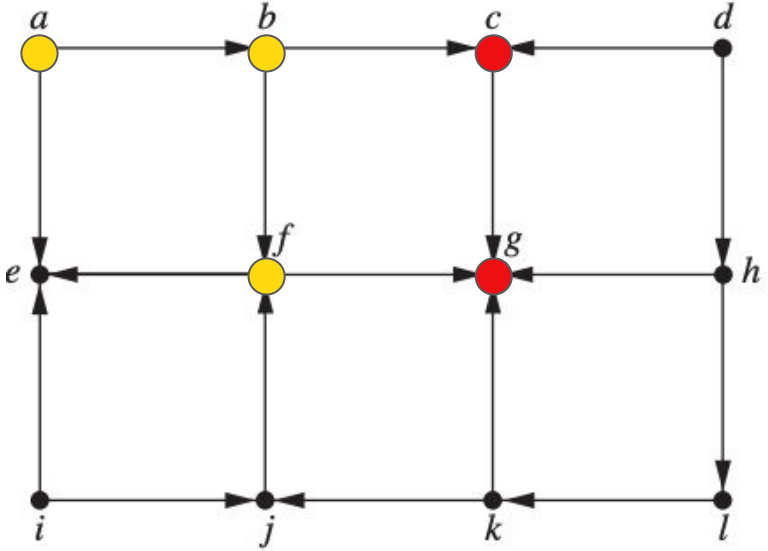


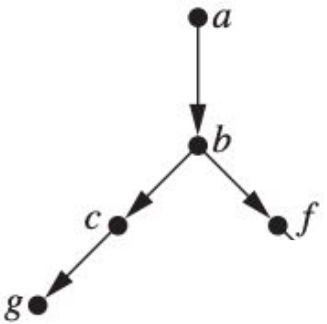


Visitado
Em visitação

DFS/BFS em grafos direcionados (7)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

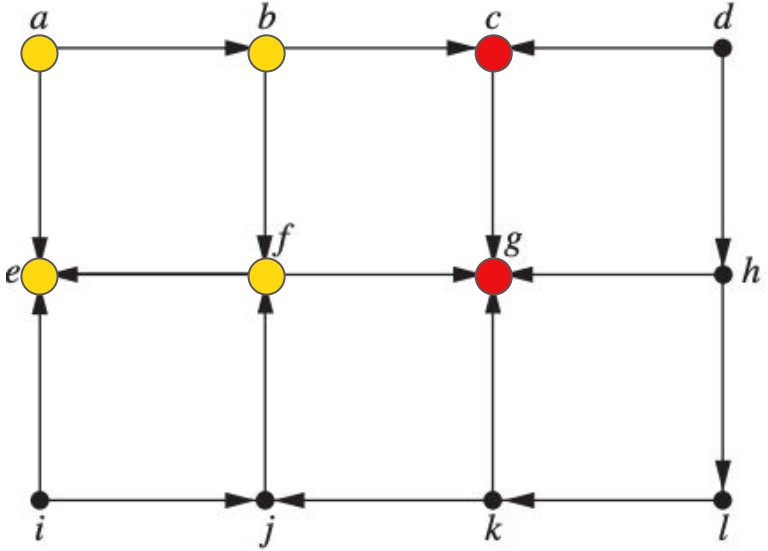


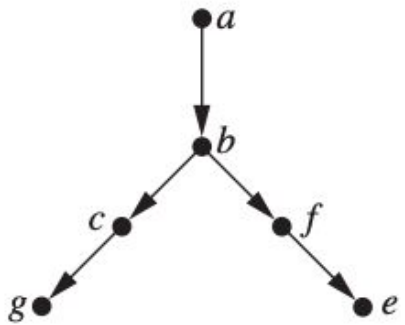


Visitado
Em visitação

DFS/BFS em grafos direcionados (8)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

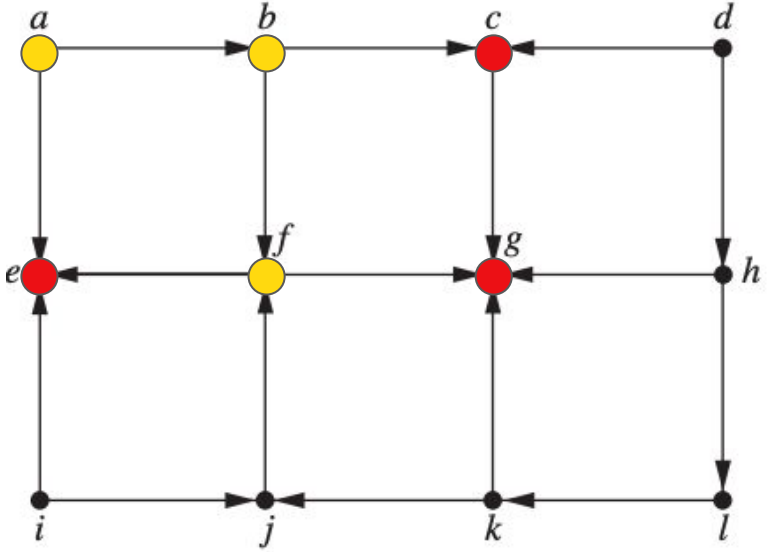


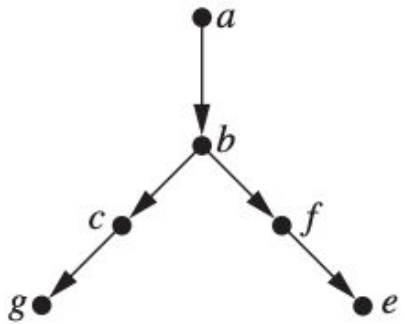


Visitado
Em visitação

DFS/BFS em grafos direcionados (9)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

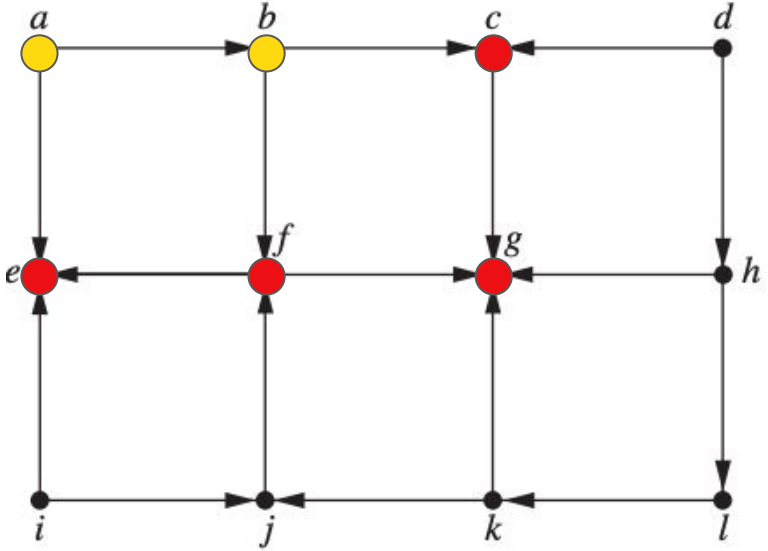


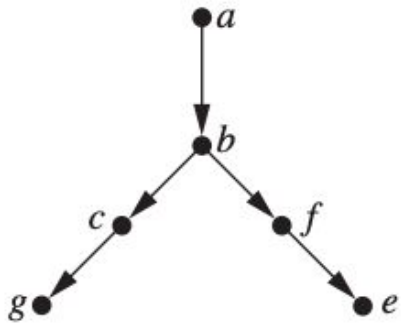


Visitado
Em visitação

DFS/BFS em grafos direcionados (10)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

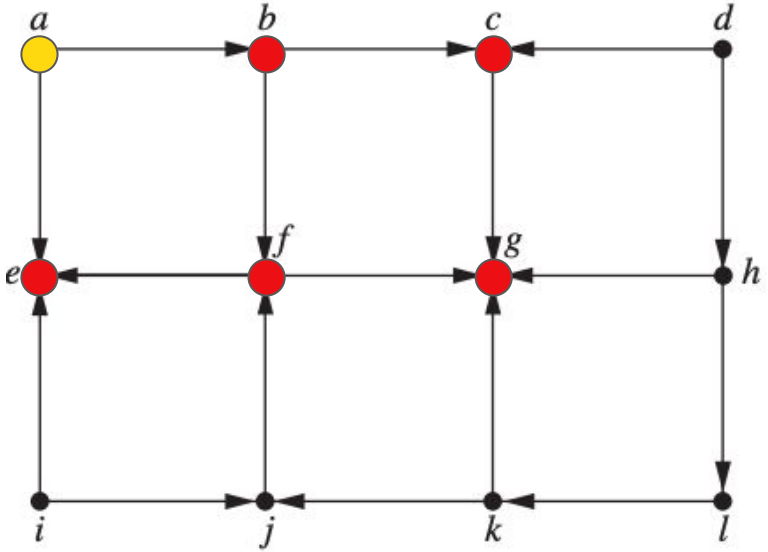


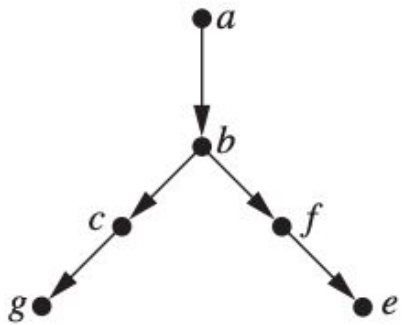


Visitado
Em visitação

DFS/BFS em grafos direcionados (11)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

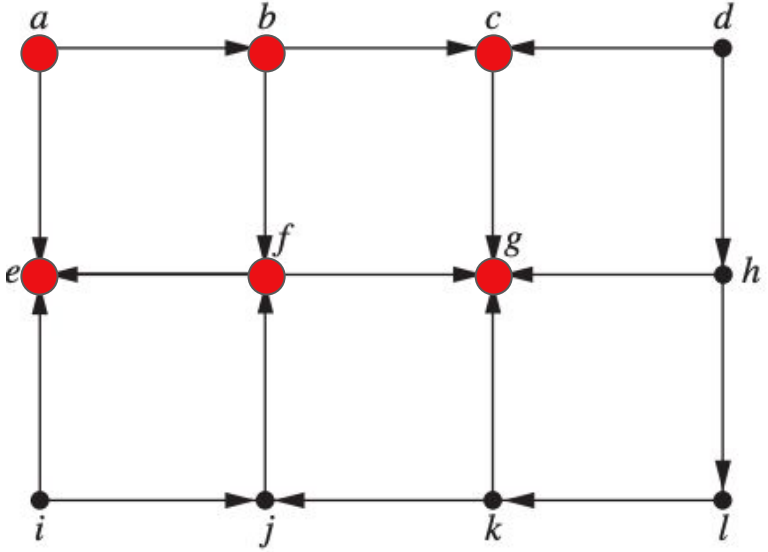


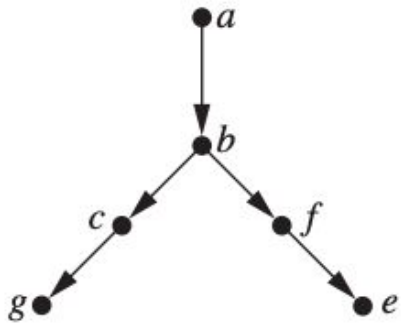


Visitado
Em visitação

DFS/BFS em grafos direcionados (12)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

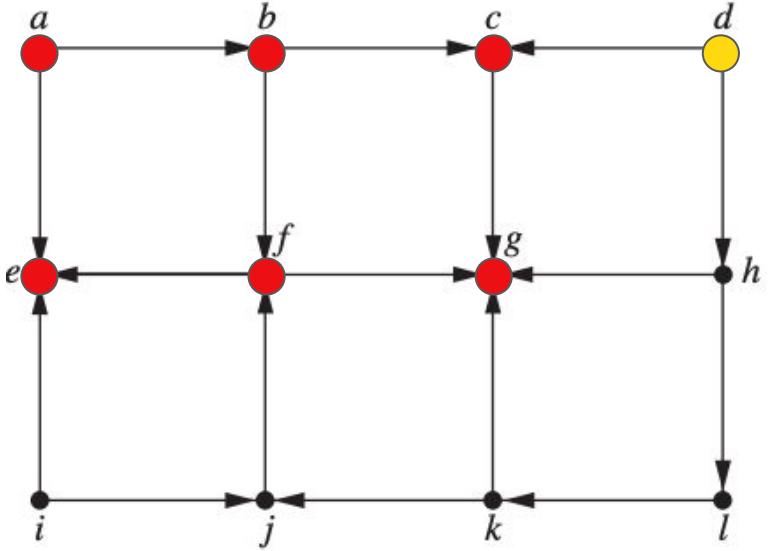


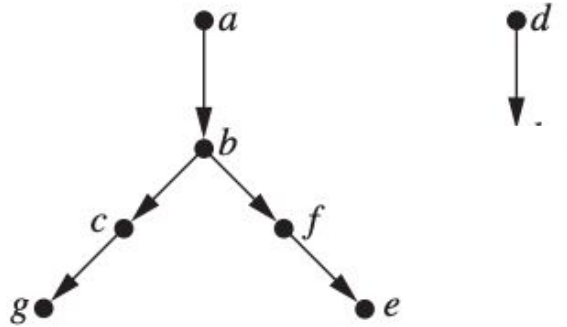


Visitado
Em visitação

DFS/BFS em grafos direcionados (13)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a

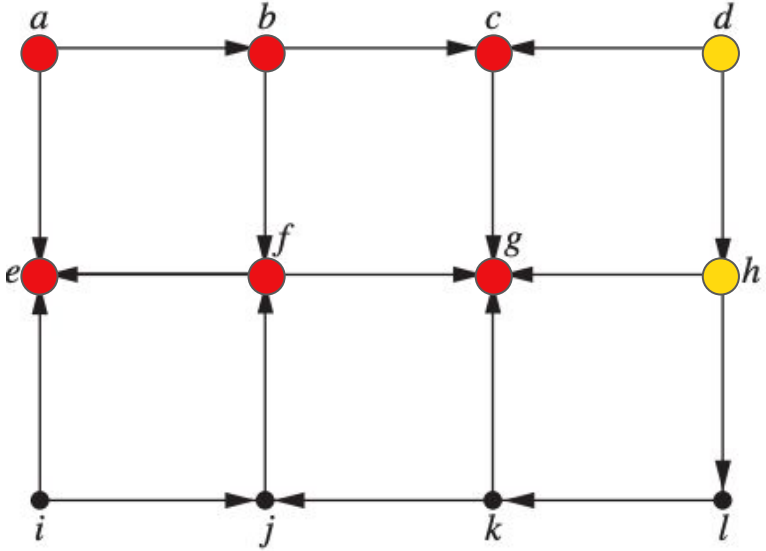


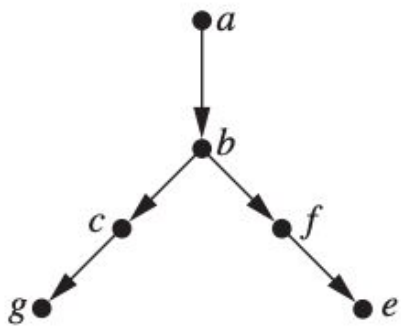


Visitado
Em visitação

DFS/BFS em grafos direcionados (14)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



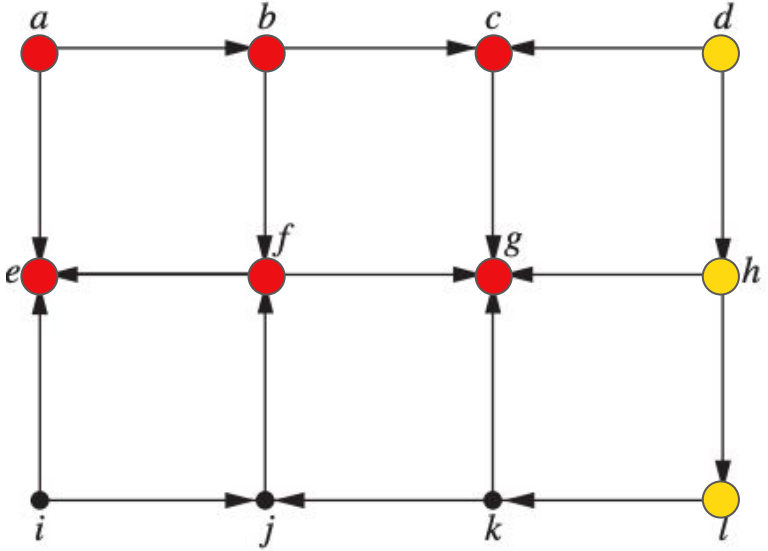


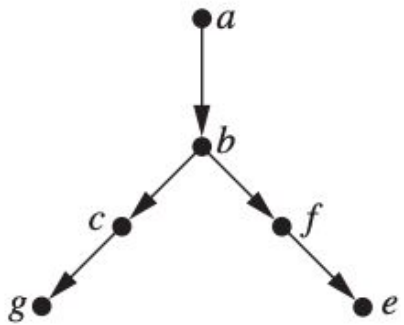


Visitado
Em visitação

DFS/BFS em grafos direcionados (15)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



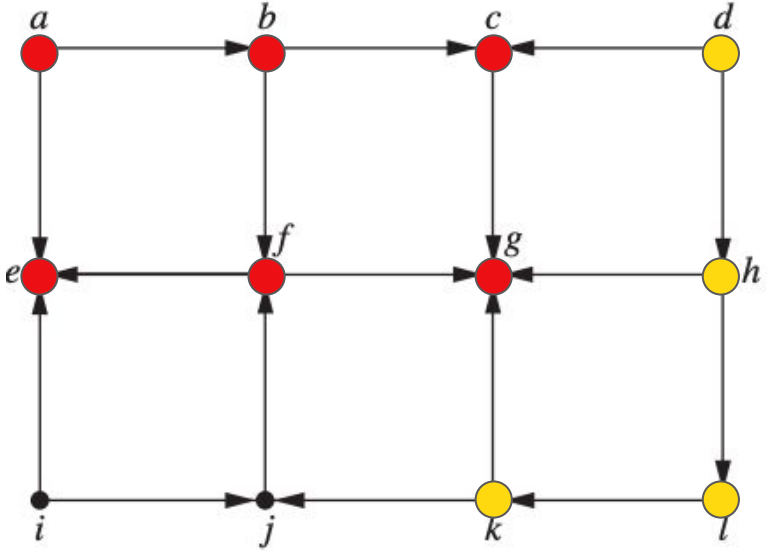


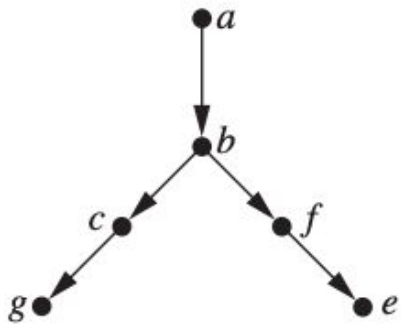


Visitado
Em visitação

DFS/BFS em grafos direcionados (16)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



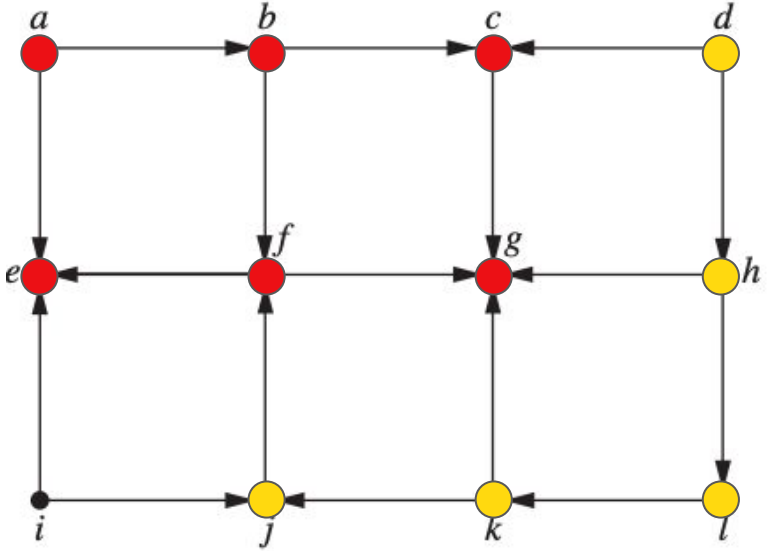


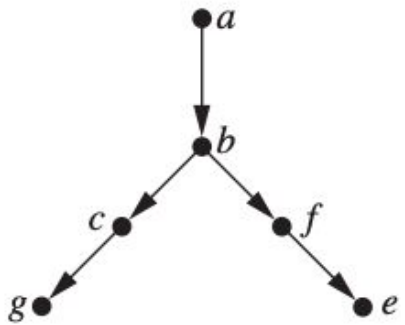


Visitado
Em visitação

DFS/BFS em grafos direcionados (17)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



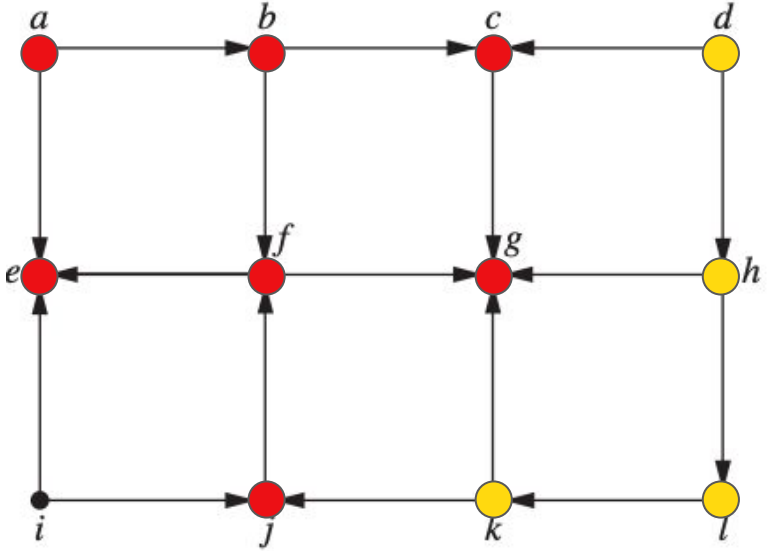


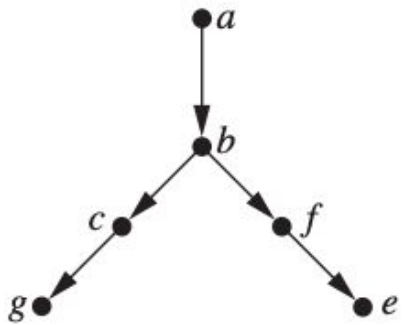


Visitado
Em visitação

DFS/BFS em grafos direcionados (18)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



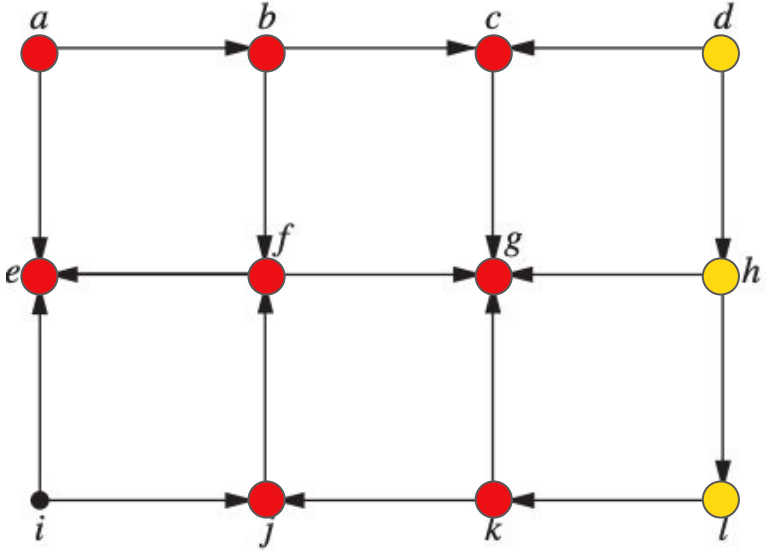


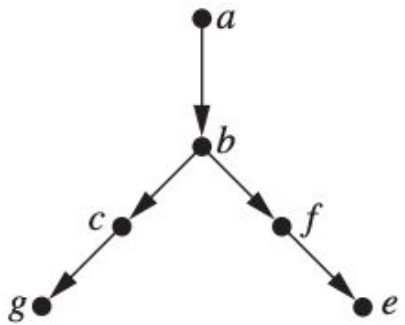


Visitado
Em visitação

DFS/BFS em grafos direcionados (19)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



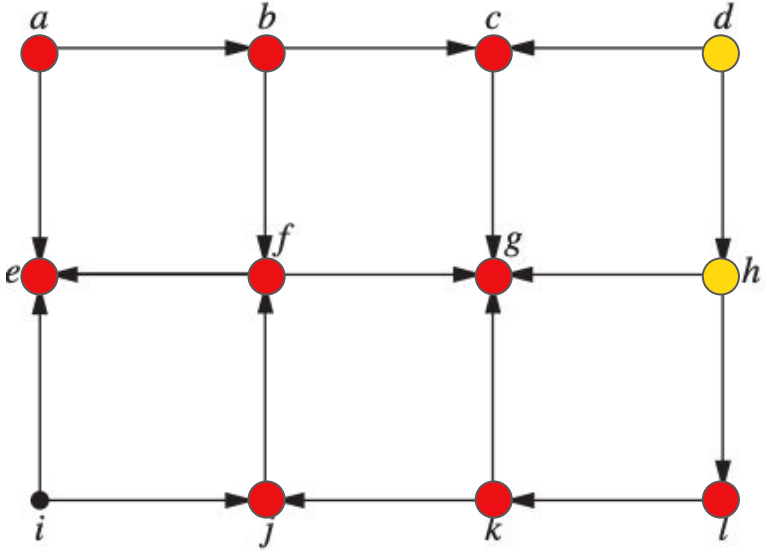


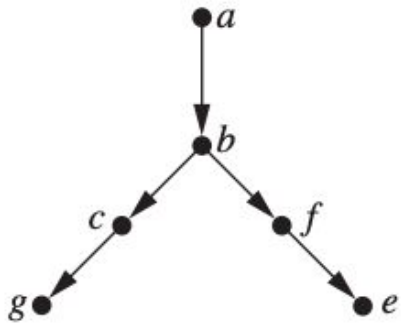


Visitado
Em visitação

DFS/BFS em grafos direcionados (20)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



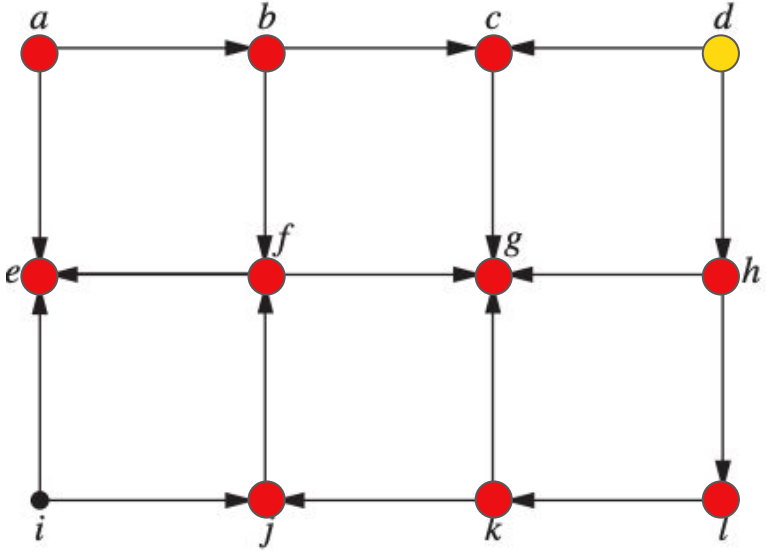


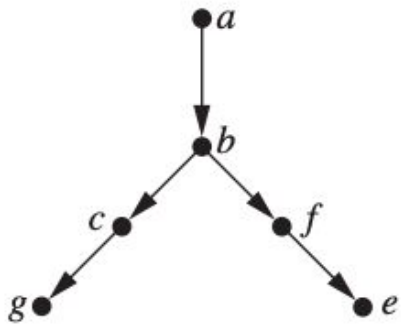


Visitado
Em visitação

DFS/BFS em grafos direcionados (21)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



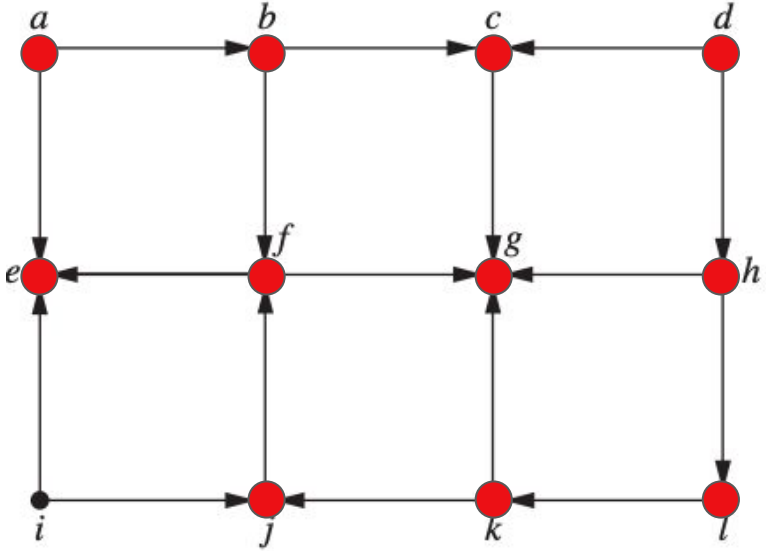


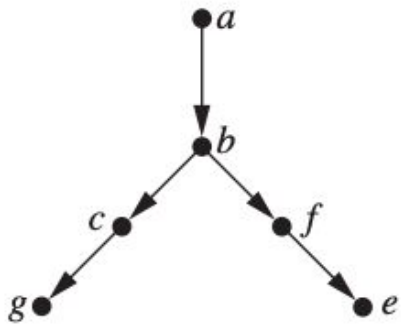


Visitado
Em visitação

DFS/BFS em grafos direcionados (22)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a



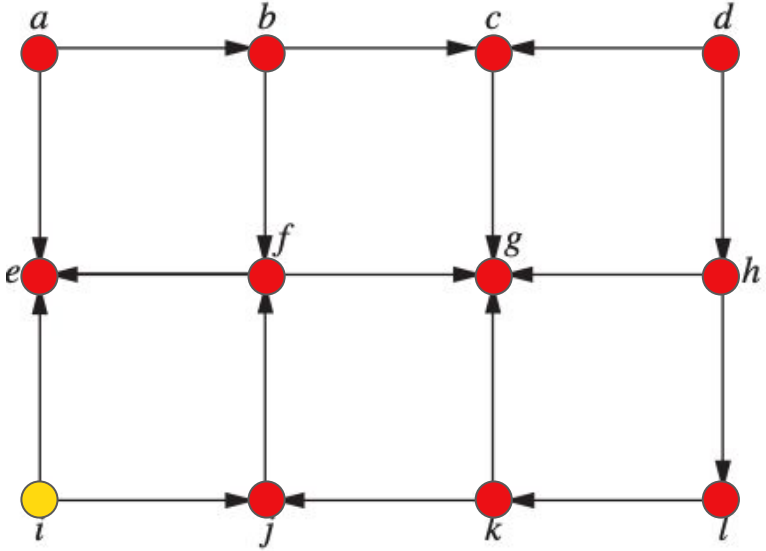


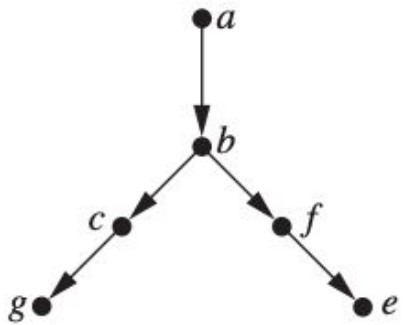


Visitado
Em visitação

DFS/BFS em grafos direcionados (23)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a





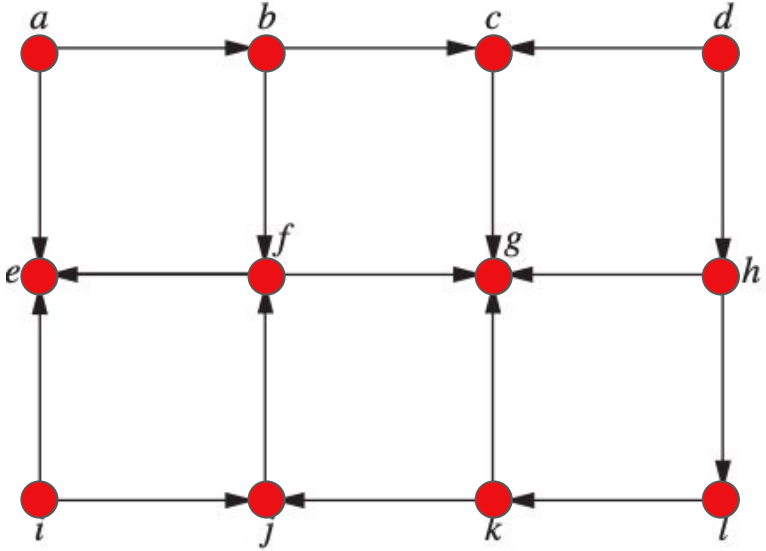


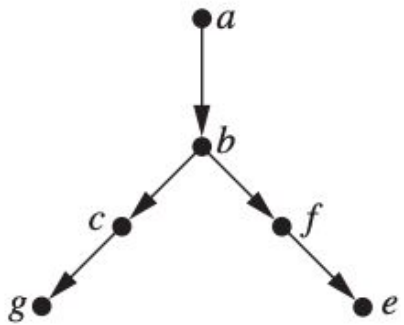


Visitado
Em visitação

DFS/BFS em grafos direcionados (24)

- Problema: determinar o resultado da busca em profundidade (DFS) Início no vértice a









Visitado
Em visitação

Teste os algoritmos

- Link